Model Compression

김장호
국민대학교 인공지능학부
jangho.kim@kookmin.ac.kr

Index

- Model compression
- Knowledge distillation (KD)
  - Online / Offline logit-based KD & Offline feature-based KD
    - Code : https://github.com/Jangho-Kim/Exercise/tree/main/Exercise_KD
- Pruning
  - Traditional pruning / Dynamic pruning
    - Code : https://github.com/Jangho-Kim/Exercise/tree/main/Exercise_pruning

Code의 경우 Pytorch 기반으로 작성되어 있음

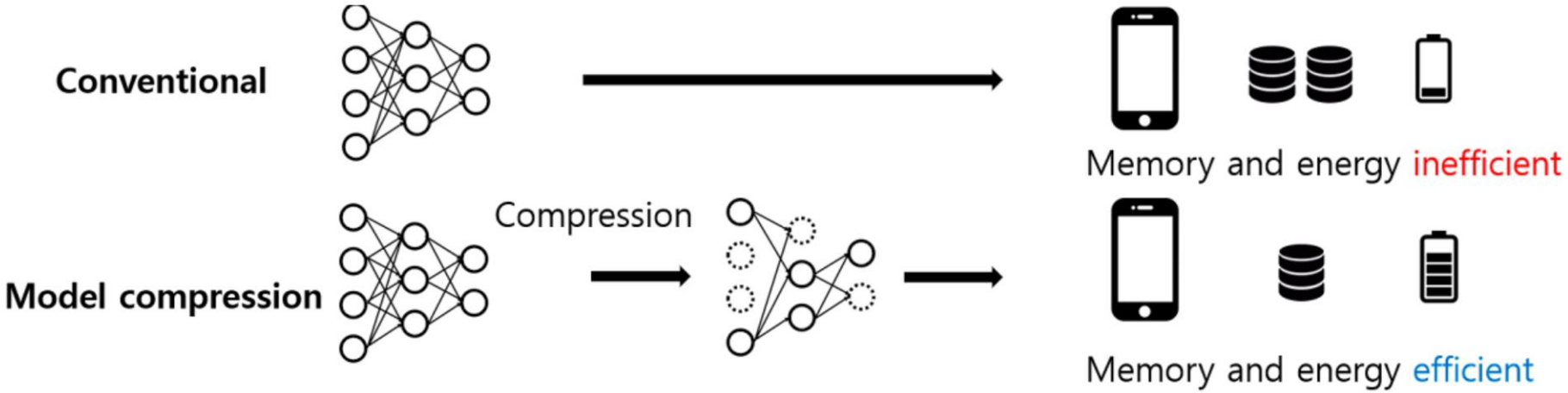
- Pytorch 기반 튜토리얼
  - https://tutorials.pytorch.kr/

- Convolution network를 활용한 이미지 분류 문제
  - https://github.com/pytorch/examples/tree/main/mnist

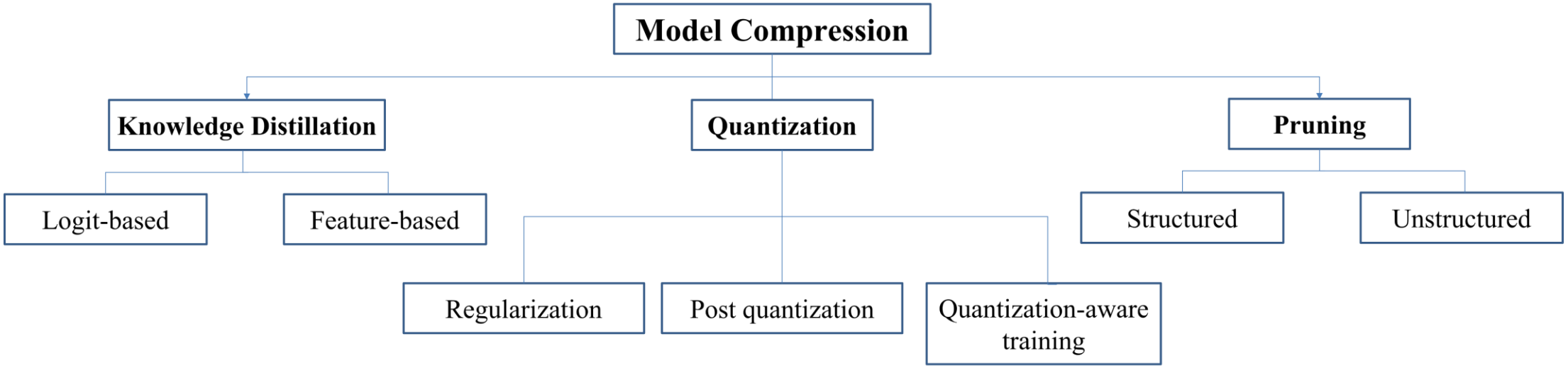
Model Compression
(모델 경량화)

모델 압축 (Model compression)

- Deep neural nets (DNNs) have shown their remarkable capabilities in various parts of computer vision and pattern recognition tasks
- DNNs generally require a vast amount of computational power and storage, which makes them difficult to be used in embedded systems that have limited resources
- Many researchers have sought ways of model compression



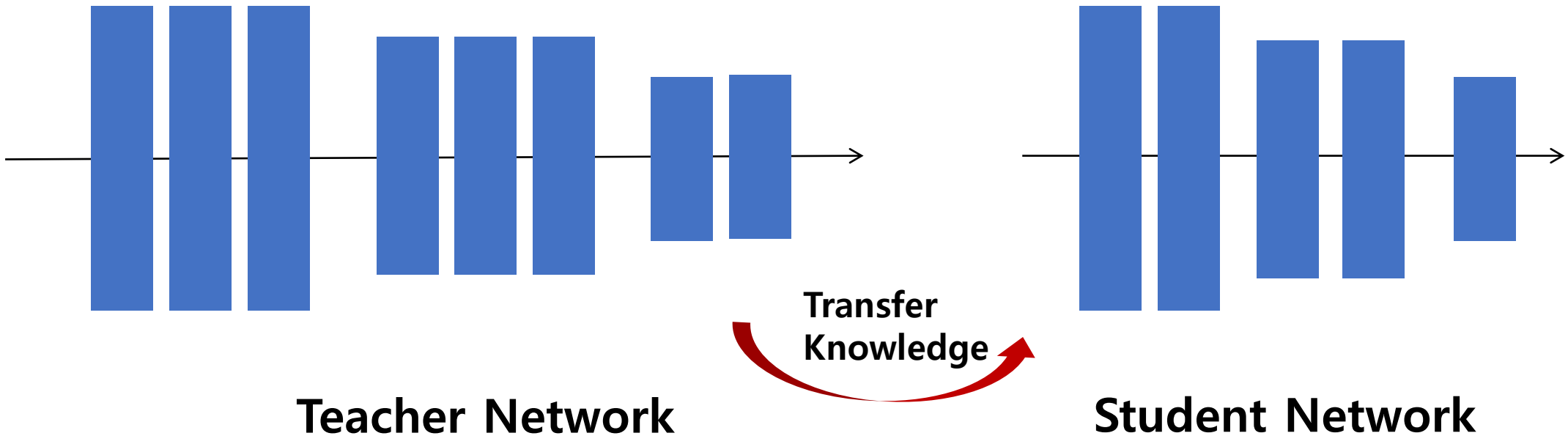
Model compression domains



* 더 많은 분야가 존재함. Ex) efficient architecture design, Low-rank factorization

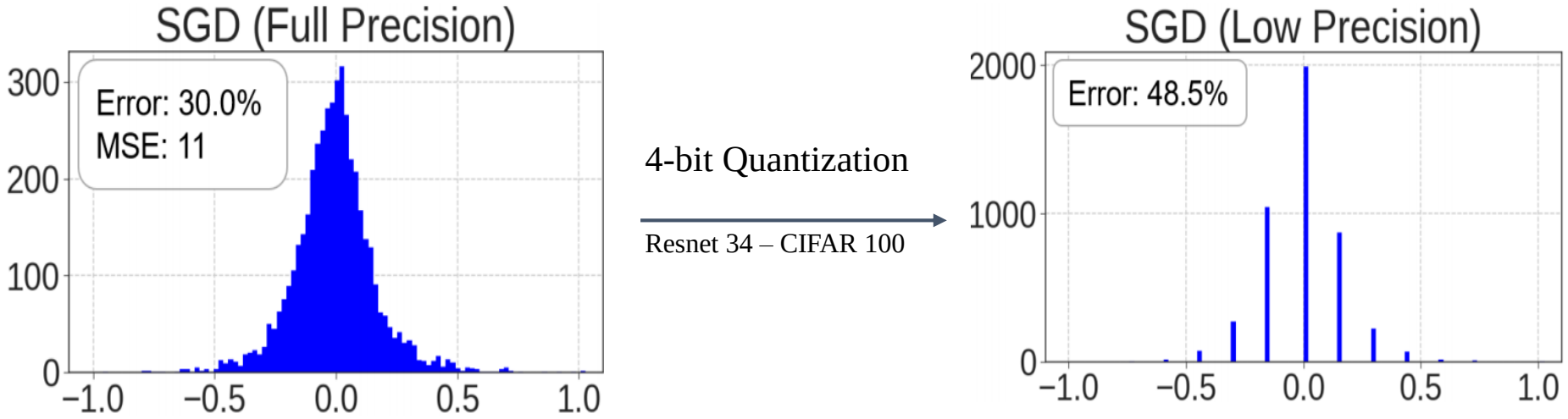
Knowledge Distillation

- Knowledge Transfer (KT) method is to transfer large model's information to a smaller network
- Student network can achieve powerful performance (accuracy) with same resou rce



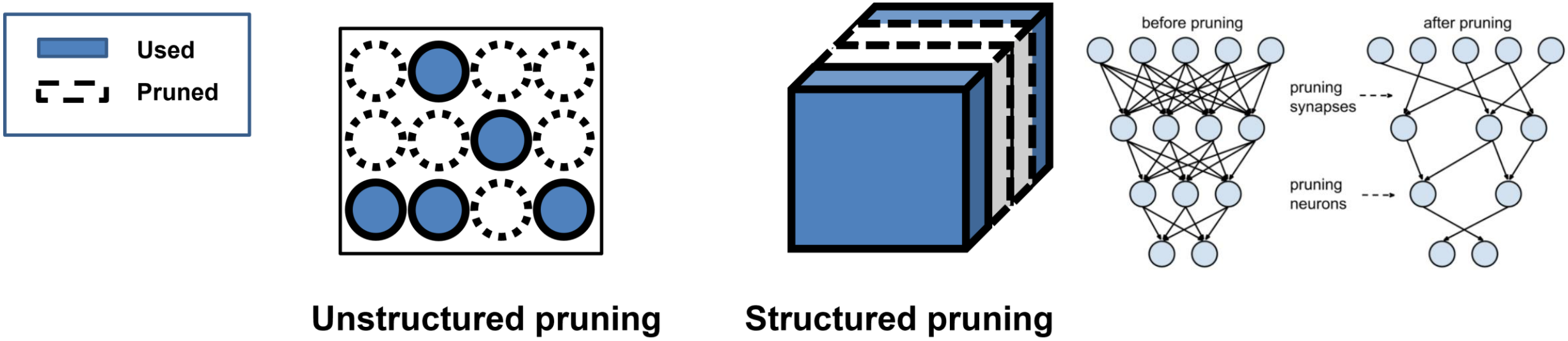
Quantization

- Quantization method quantizes floating point values into discrete values to approximate them by a set of integers and scaling factors
- Quantization allows for more power-efficient operations and convolution computations at the expense of lower bit-width representation



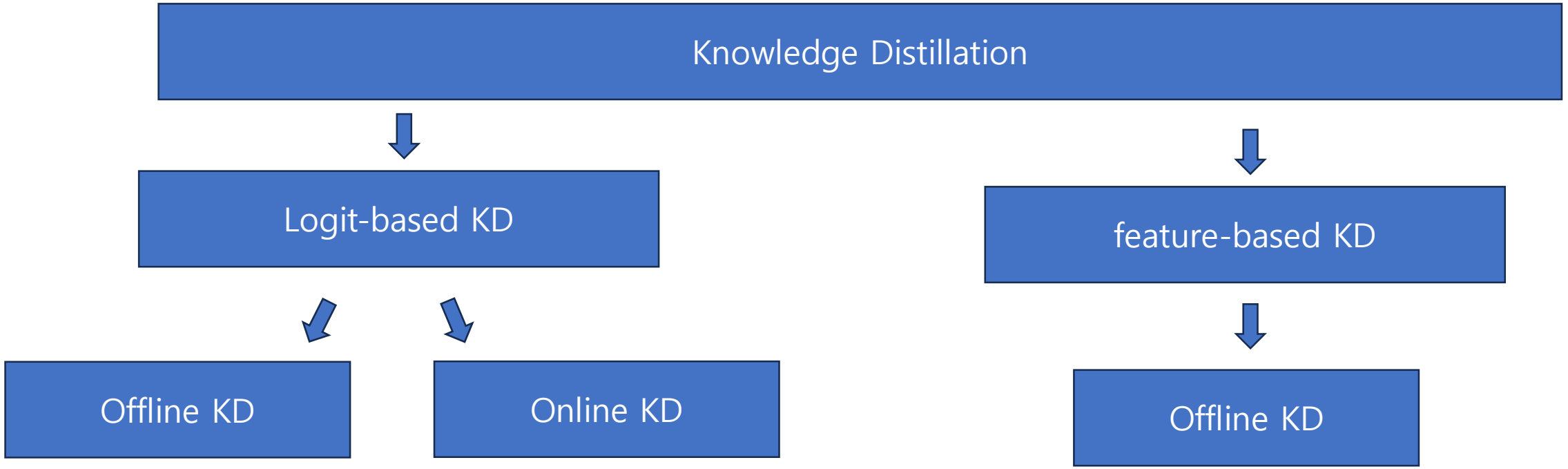
Pruning

- Pruning method prunes the unimportant weights or channels based on different criteria
- Pruning method can reduce model memory and the number of flops by eliminating unimportant weights or channels



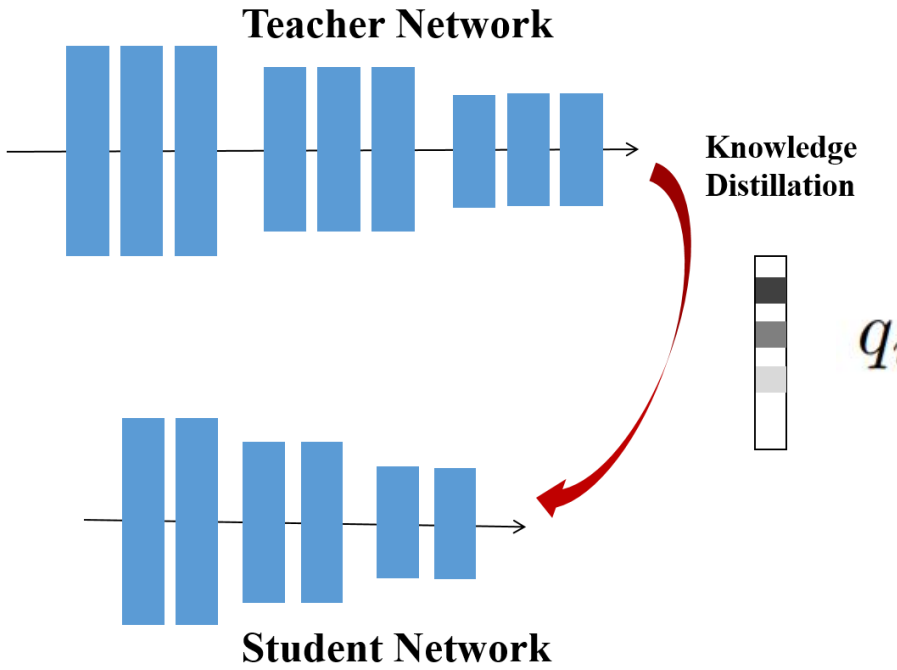
Knowledge Distillation
(지식 증류)

Knowledge Distillation



Knowledge Distillation (logit-based KD)

- Knowledge Distillation [1] is a transferring method that aims to improve training of student network by relying on knowledge borrowed from a powerful teacher network

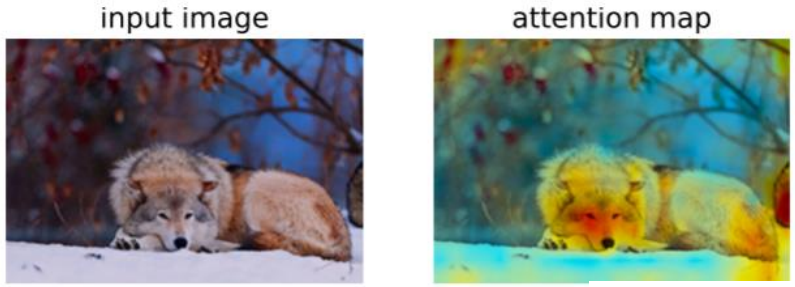


$$q_i = \frac{exp(z_i/T)}{\sum_j exp(z_j/T)}$$

Z is a logit and q is a probability

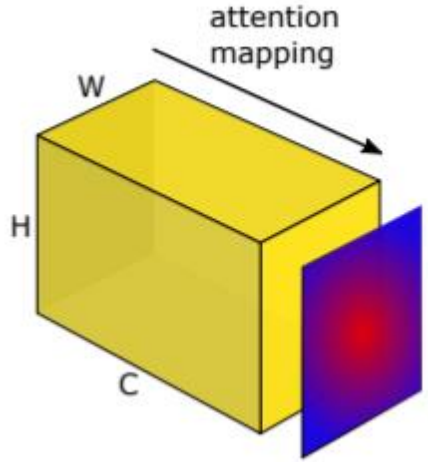
[1] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015

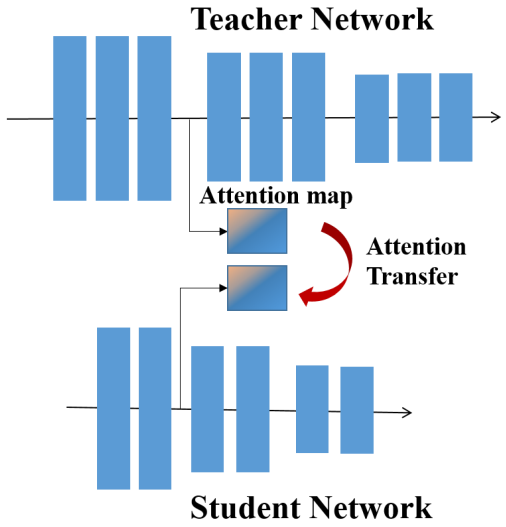
Attention transfer (Feature-based KD)



In attention transfer [2] , consider the following activation-based spatial attention map.

$$F^p_{\text{sum}}(A) = \sum_{i=1}^{C} |A_i|^p$$





[2] Sergey Zagoruyko and Nikos Komodakis. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. arXiv preprint arXiv:1612.03928, 2016

Logit-based KD

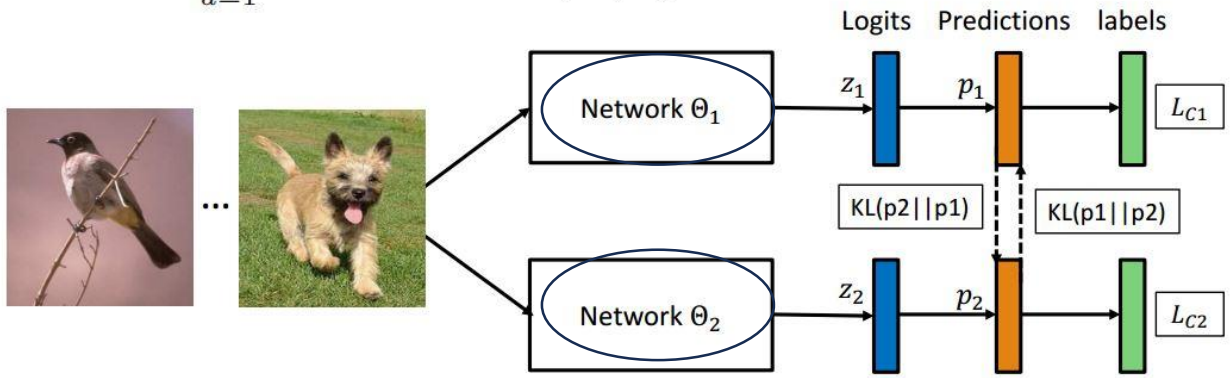
- Online KD의 경우 teacher와 student 모두 학습이 되지 않은 상태에서 시작
- Offline KD의 경우 teacher는 pre-trained된 model을 사용함

Online KD

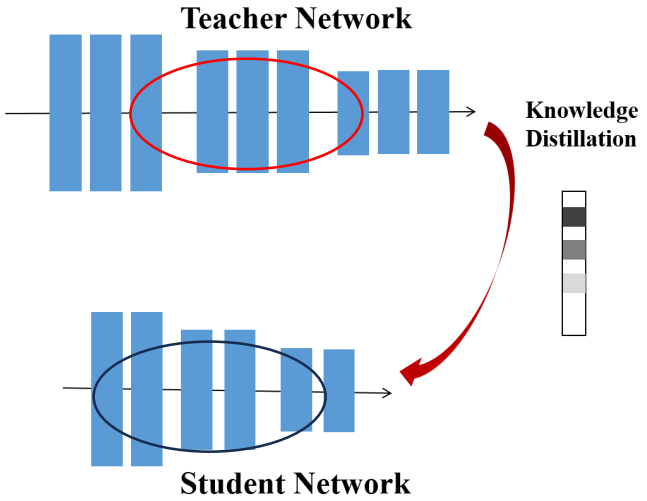
$$KL(\mathbf{z}^T||\mathbf{z}^S;\mathcal{T}) = \sum_{a=1}^{m} \sigma_a(\mathbf{z}^T;\mathcal{T}) \log\left(\frac{\sigma_a(\mathbf{z}^T;\mathcal{T})}{\sigma_a(\mathbf{z}^S;\mathcal{T})}\right)$$

$$\mathcal{L}_{KD}^{S} = \alpha\mathcal{L}_{ce}^{S} + \beta(\mathcal{T}^2 * KL(\mathbf{z}^T||\mathbf{z}^S;\mathcal{T}))$$

$$\mathcal{L}_{KD}^{T} = \alpha\mathcal{L}_{ce}^{T} + \beta(\mathcal{T}^2 * KL(\mathbf{z}^S||\mathbf{z}^T;\mathcal{T}))$$

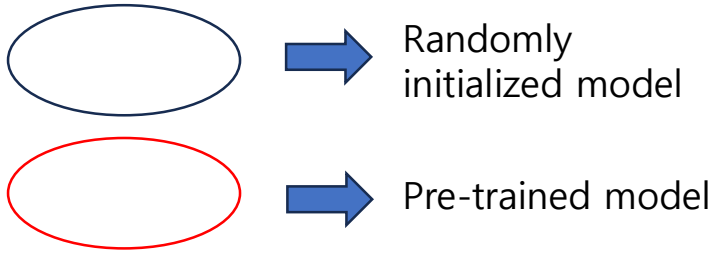


Offline KD



Deep mutual learning [3]

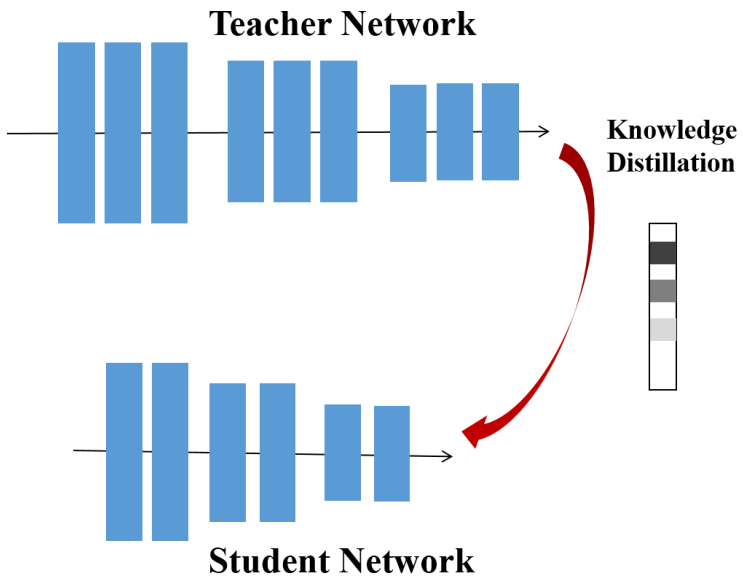
Knowledge Distillation



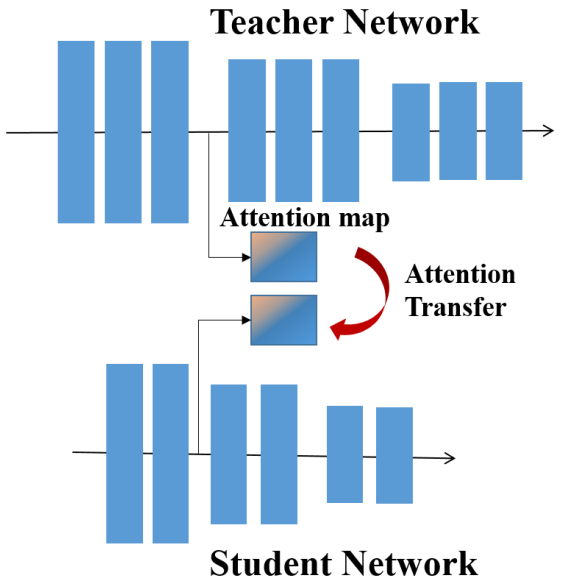
[3] Zhang, Ying, et al. "Deep mutual learning." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018.

FT – Introduction (Feature-based offline KD)

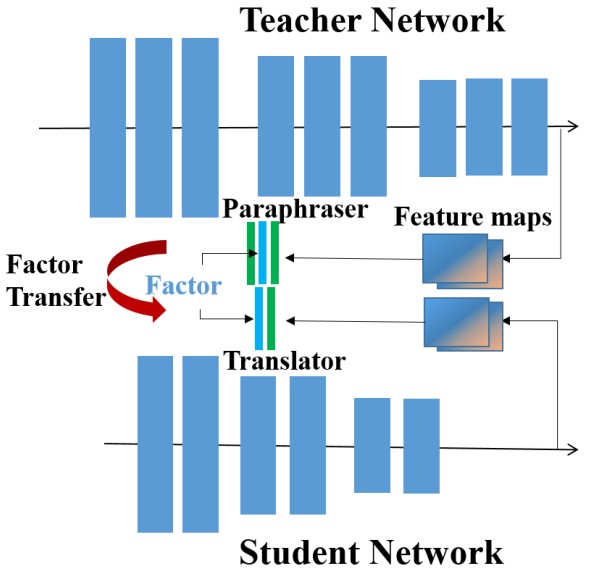
- Existing methods transfer knowledge of teacher network to the student network directly



<Knowledge Distillation>



<Attention Transfer>



<Factor Transfer (proposed)>

FT – Introduction

- Directly transferring the teacher's outputs overlooks the inherent differences between the teacher network and the student network, such as the network structure, the number of channels, and initial conditions.

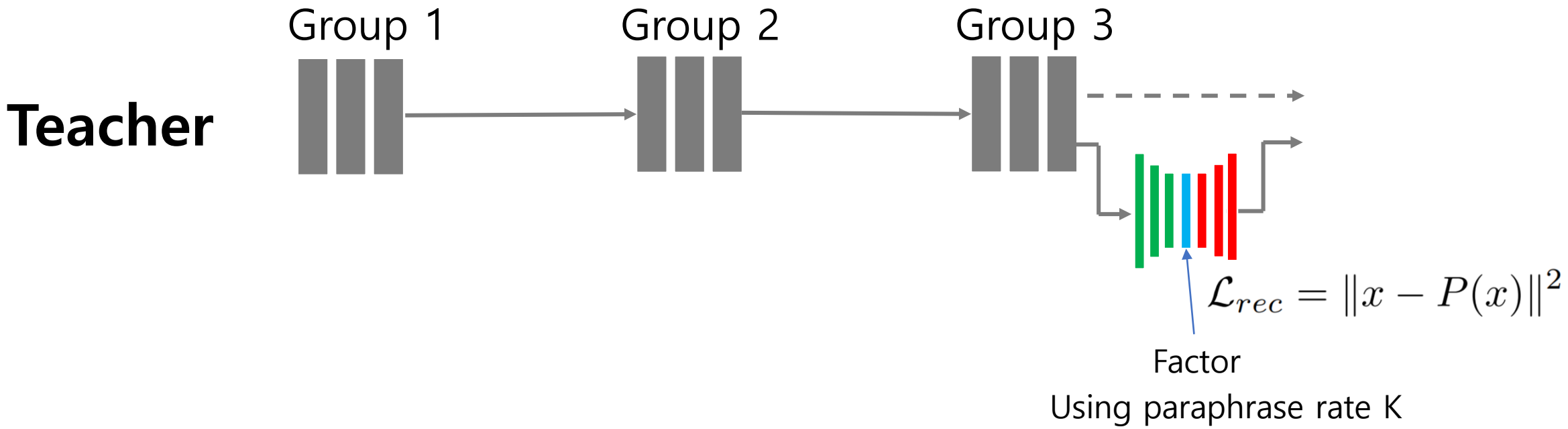
- Unlike KD and AT, Factor Transfer (FT) [4] does not directly compare the outputs. FT extracts **factors** from both the teacher and the student network, whose difference is tried to be minimized.

- This is done by two convolutional modules, which are called a paraphraser and a translator.

[4] Kim, Jangho, SeongUk Park, and Nojun Kwak. "Paraphrasing complex network: Network compression via factor transfer." *Advances in neural information processing systems* 31 (2018).

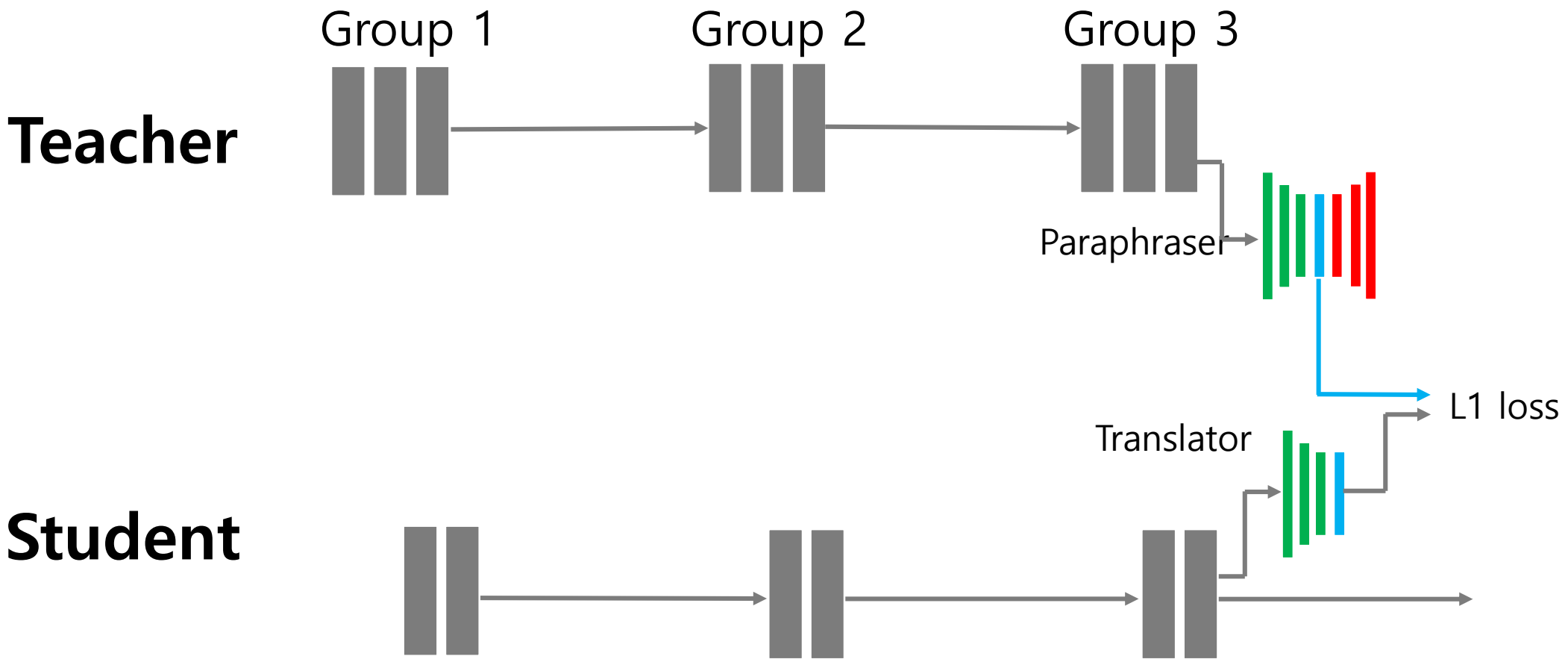
FT – Proposed method

- Training the paraphraser using the auto-encoder structure



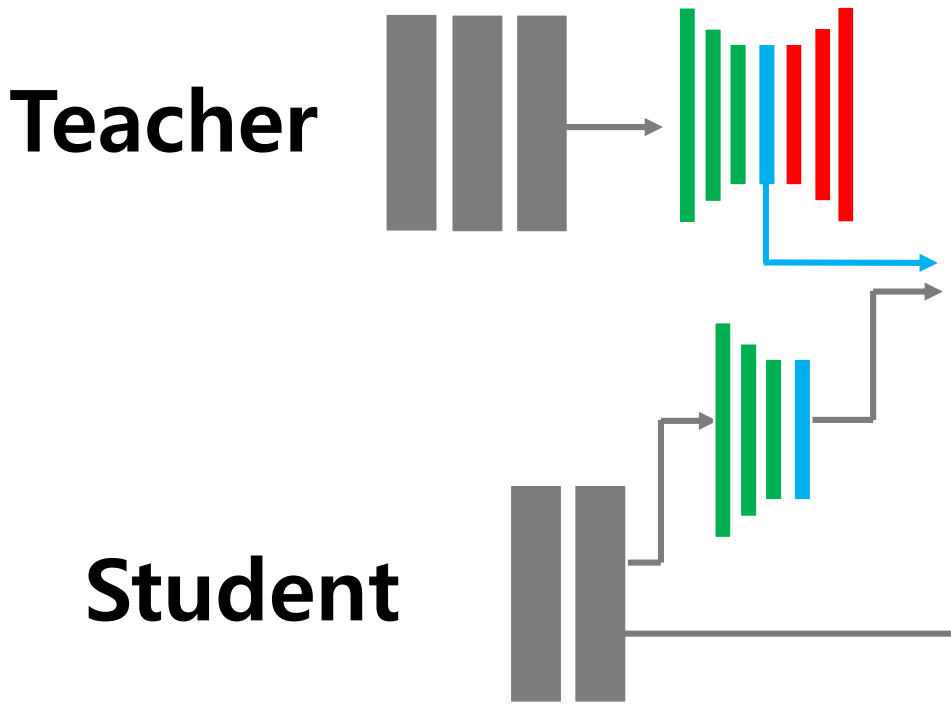
FT – Proposed method

- Training a student network



FT – Proposed method

The student network should be able to absorb and digest the factor on its own way



$$\mathcal{L}_{student} = \mathcal{L}_{cls} + \beta L_{FT}$$

$$\mathcal{L}_{FT} = \| \frac{F_T}{\|F_T\|_2} - \frac{F_S}{\|F_S\|_2} \|_p. \quad (p = 1)$$

$$\mathcal{L}_{cls} = \mathcal{C}(S(x), y)$$

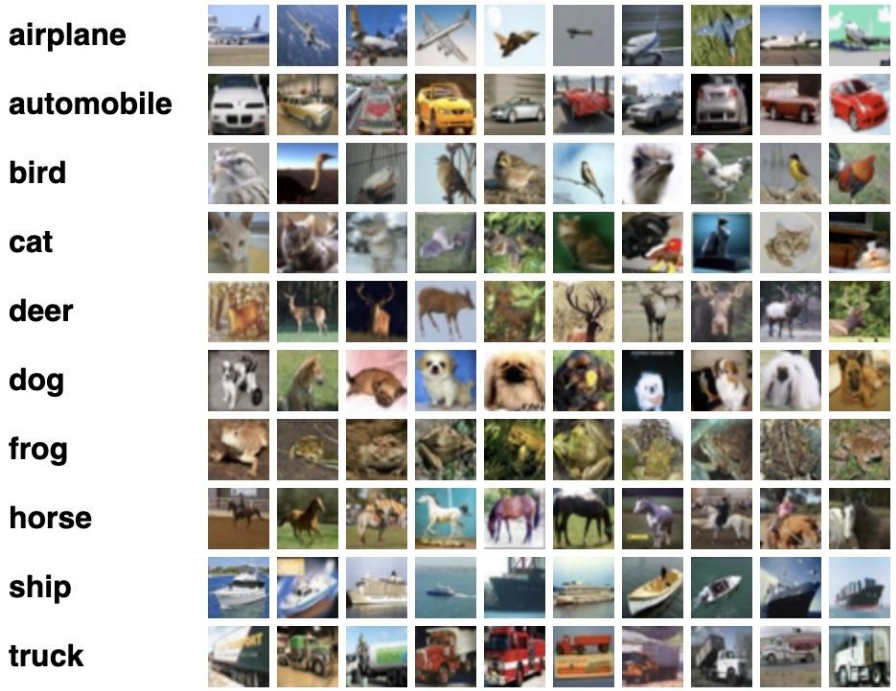
Exercise_KD (Code 실습 1)

- 1. offline KD – KD.py
- 2. online KD – DML.py
- 3. offline feature distillation – FT.py

*Code 설명은 동영상 참고
https://github.com/Jangho-Kim/Exercise/tree/main/Exercise_KD

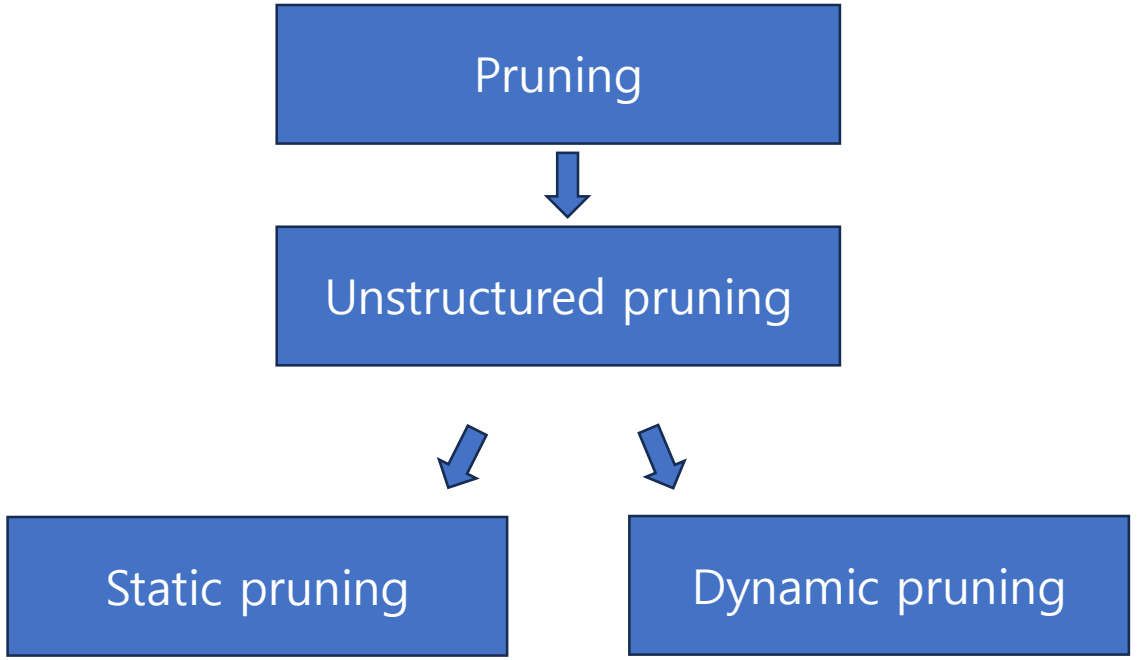
Datasets

- CIFAR 10/100 – Image classification task
  - CIFAR100 - CIFAR-100 dataset contains the same number of images as CIFAR-10 dataset, 50K (train) and 10K (test), but has 100 classes, containing only 500 images per classes.



Pruning (가지 치기)

Pruning


Pruning
Unstructured pruning
Static pruning
Dynamic pruning

Gradient descent (경사하강법)

Gradient Descent Algorithm:

Pick an initial point w_0

Iterate until convergence

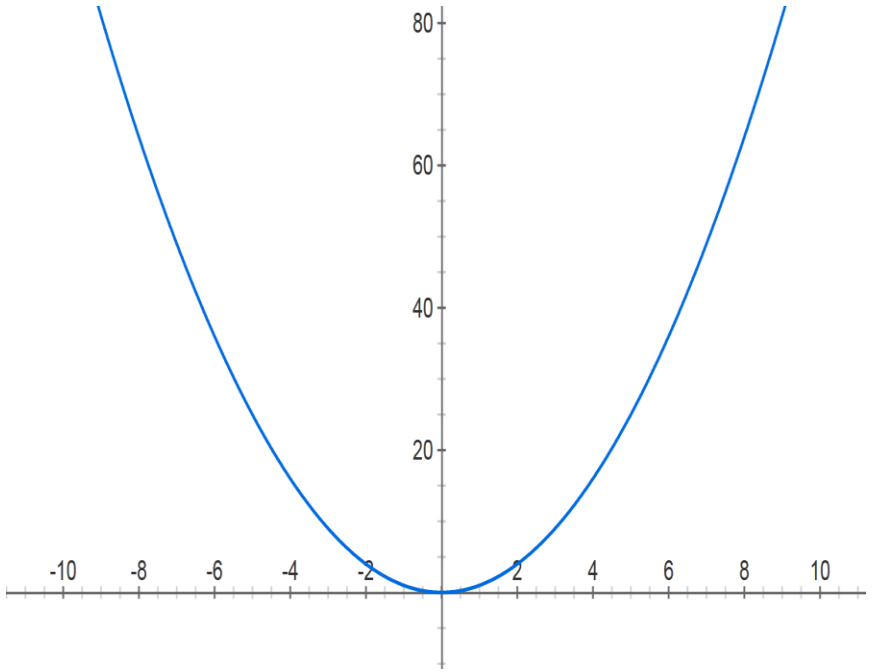
$$w_{t+1} = w_t - lr\nabla f(w_t)$$

where lr is the tth step size (sometimes called learning rate)

$f(w) = w^2$

$w^{(0)} = -4$

$w^{(1)} = -4 - .8 \cdot 2 \cdot (-4)$

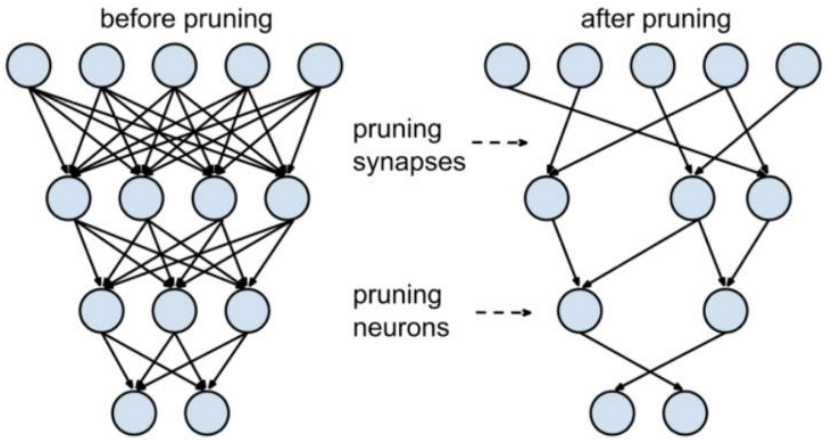


Pruning (static pruning)

- Static pruning just prunes the model once with specific sparsity after model training and update active weights [5]

$$\overline{w}_i \leftarrow \overline{w}_i - lr\frac{\partial \mathcal{L}}{\partial \overline{w}_i}, \ \ \forall i \in \{1, \ldots, P\}$$

Pruning criteria : weight magnitude



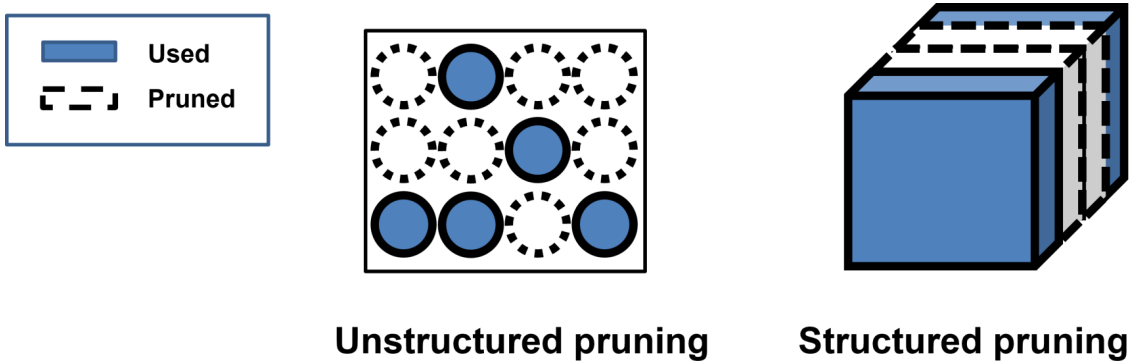
where i is the index of weights and mask and $\overline{w}_i = m_i \times w_i$. Sparse weights consist of two mutually exclusive sets which are active (unpruned) ($\{w_i | w_i = \overline{w}_i, m_i = 1\}$) and inactive (pruned) ($\{w_i | w_i \neq \overline{w}_i, \ \overline{w}_i = m_i = 0\}$). Many

[5] Han, Song, et al. "Learning both weights and connections for efficient neural networks." NeurIPS (2015).

Pruning (dynamic pruning)

- Dynamic pruning updates active weights and inactive weights for dynamic model structure [6]

$$w_i \leftarrow w_i - lr \frac{\partial \mathcal{L}}{\partial \overline{w}_i} \frac{\partial \overline{w}_i}{\partial w_i}, \forall i \in \{1, \ldots, P\}$$



where i is the index of weights and mask and $\overline{w}_i = m_i \times w_i$. Sparse weights consist of two mutually exclusive sets which are active (unpruned) ($\{w_i | w_i = \overline{w}_i, m_i = 1\}$) and inactive (pruned) ($\{w_i | w_i \neq \overline{w}_i, \overline{w}_i = m_i = 0\}$). Many

[6] Lin, Tao, et al. "Dynamic model pruning with feedback." ICLR (2020).

Exercise_pruning (Code 실습 2)

- 1. Static pruning – Static_pruning.py // ./Static.sh
- 2. Dynamic pruning – Dynamic_pruning.py // ./Dynamic.sh

*Code 설명은 동영상 참고
https://github.com/Jangho-Kim/Exercise/tree/main/Exercise_pruning

$$S_c = S_t + (S_i - S_t)(1 - \frac{c - c_0}{n})^3. \quad (5.1)$$

We gradually increase the sparsity ratio from an initial sparsity ratio (S_i at $c_0 = 0$) based on the current epoch c to the target sparsity ratio (S_t). S_c means the current sparsity at c, where $c \in \{c_0, ..., c_0 + n\}$ and n is the number of epochs for the overall training.

Pruning schedule

Question



Appendix

DCIL – Introduction (ACM MM 2023, accecpted)

- Problem
  - Many works have resorted to using a form of coarse gradient, in which the derivative of the masked weight with respect to the real weight is simply approximated to 1
  - While using this form of coarse gradient is a convenient practice to revive the inactive weights, it may update the weights to an unintended direction causing harm to the training stability and performance.

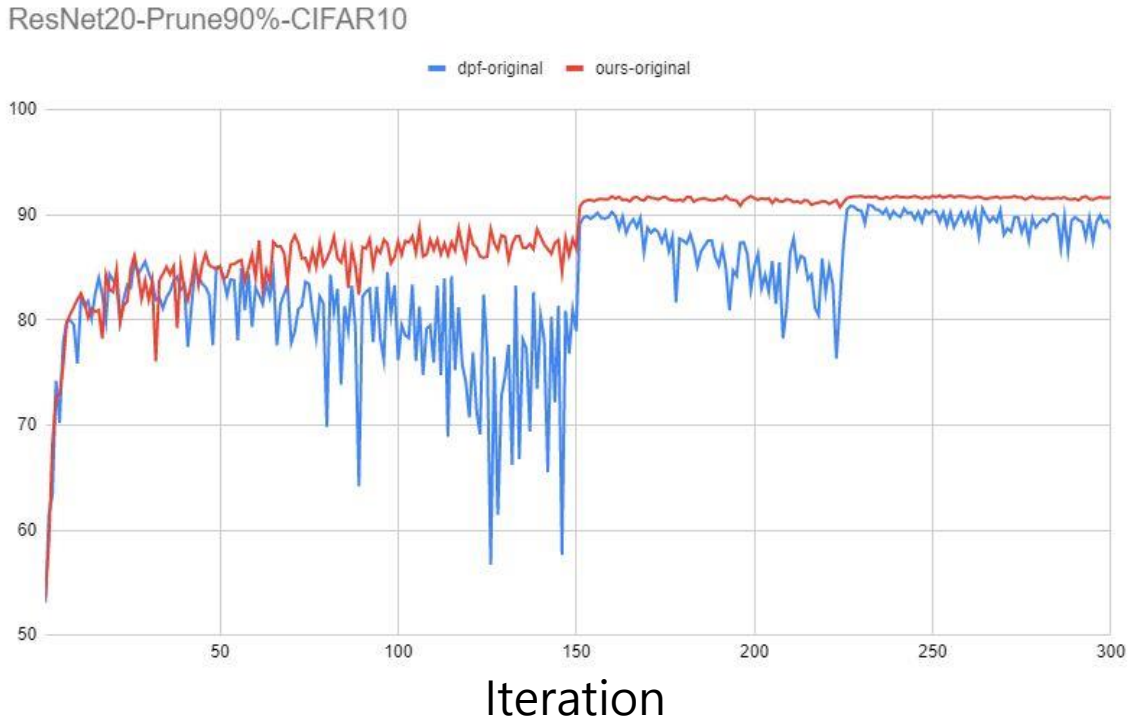
[3] Dynamic pruning with feedback, Tao lin et al, 2020 ICLR

$$w_i \leftarrow w_i - \frac{\partial \mathcal{L}}{\partial \overline{w}_i} \frac{\partial \overline{w}_i}{\partial w_i}, \forall i \in \{1, \ldots, P\}$$

$\frac{\partial \overline{w}_i}{\partial w_i} = 1$

where i is the index of weights and mask and $\overline{w}_i = m_i \times w_i$. Sparse weights consist of two mutually exclusive sets which are active (unpruned) ($\{w_i | w_i = \overline{w}_i, m_i = 1\}$) and inactive (pruned) ($\{w_i | w_i \neq \overline{w}_i,\ \overline{w}_i = m_i = 0\}$).

DCIL – Introduction



$$w_i \leftarrow w_i - \frac{\partial \mathcal{L}}{\partial \overline{w}_i} \frac{\partial \overline{w}_i}{\partial w_i}, \forall i \in \{1, \ldots, P\}$$

$\frac{\partial \overline{w}_i}{\partial w_i} = 1$

$$S_c = S_t + (S_i - S_t)(1 - \frac{c - c_0}{n})^3. \quad (5.1)$$

We gradually increase the sparsity ratio from an initial sparsity ratio (S_i at $c_0 = 0$) based on the current epoch c to the target sparsity ratio (S_t). S_c means the current sparsity at c, where $c \in \{c_0, ..., c_0 + n\}$ and n is the number of epochs for the overall training.

[3] Dynamic pruning with feedback, Tao lin et al, 2020 ICLR

DCIL – Proposed method

Let $\mathcal{F}^{\mathbf{W}} : \mathcal{X} \rightarrow \mathcal{Y}$ and $\mathcal{L} : \mathcal{Y} \rightarrow \mathbb{R}$ be the forwarding path with given weights ($\mathbf{W}$) and the loss function, respectively. Then, we can define the update rule of DCIL as follows:

- Update rules

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\{\mathbf{M} \odot \nabla_{\overline{\mathbf{w}}}\mathcal{L} + (1 - \mathbf{M}) \odot \nabla_{\mathbf{w}}\mathcal{L}\}$$

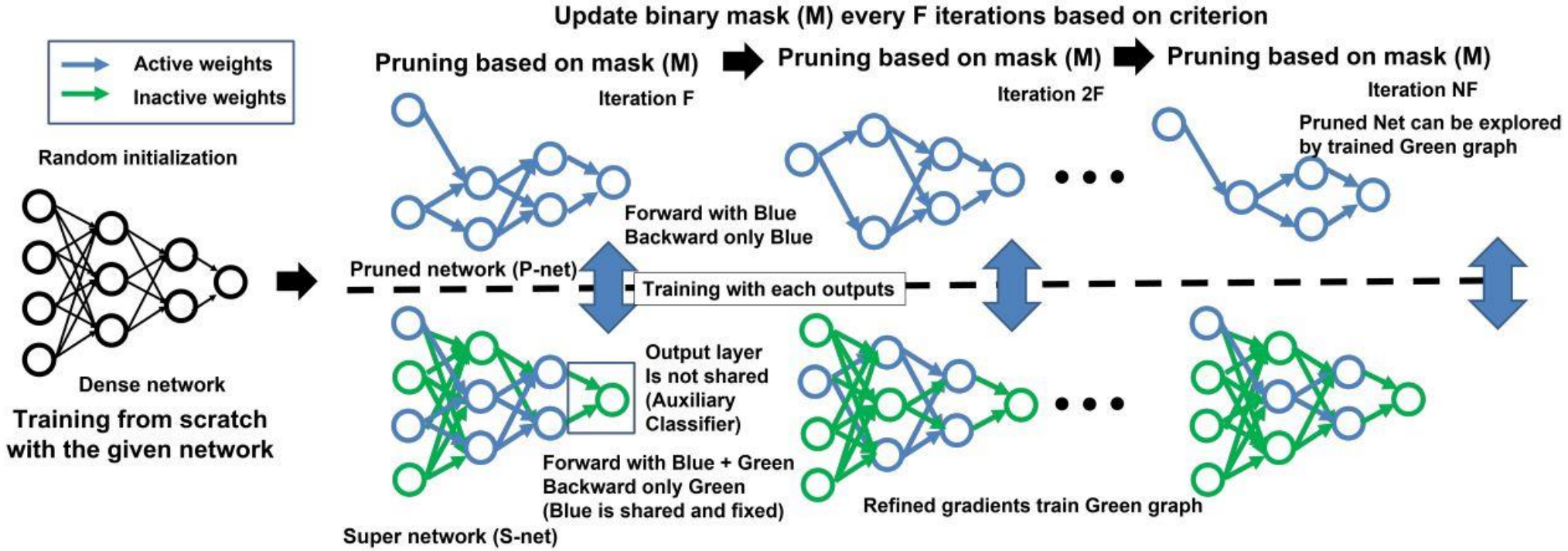
$$\nabla_{\overline{\mathbf{w}}}\mathcal{L} \triangleq \frac{\partial \mathcal{L} \circ \mathcal{F}^{\overline{\mathbf{W}}}(\mathbf{X})}{\partial \overline{\mathbf{w}}}, \quad \nabla_{\mathbf{w}}\mathcal{L} \triangleq \frac{\partial \mathcal{L} \circ \mathcal{F}^{\mathbf{W}}(\mathbf{X})}{\partial \mathbf{w}}$$

DCIL – Proposed method

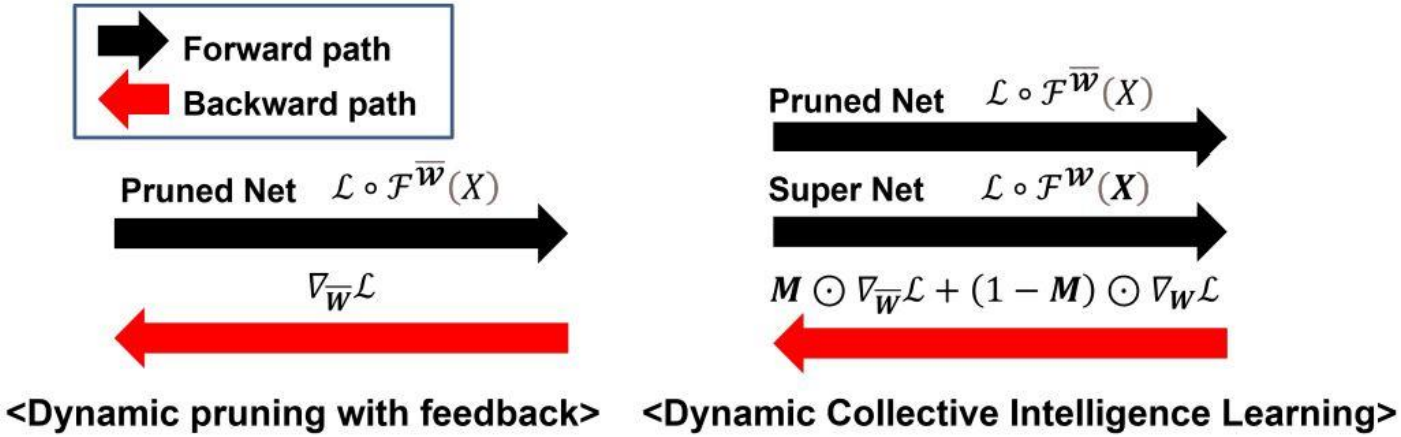
- Update rules

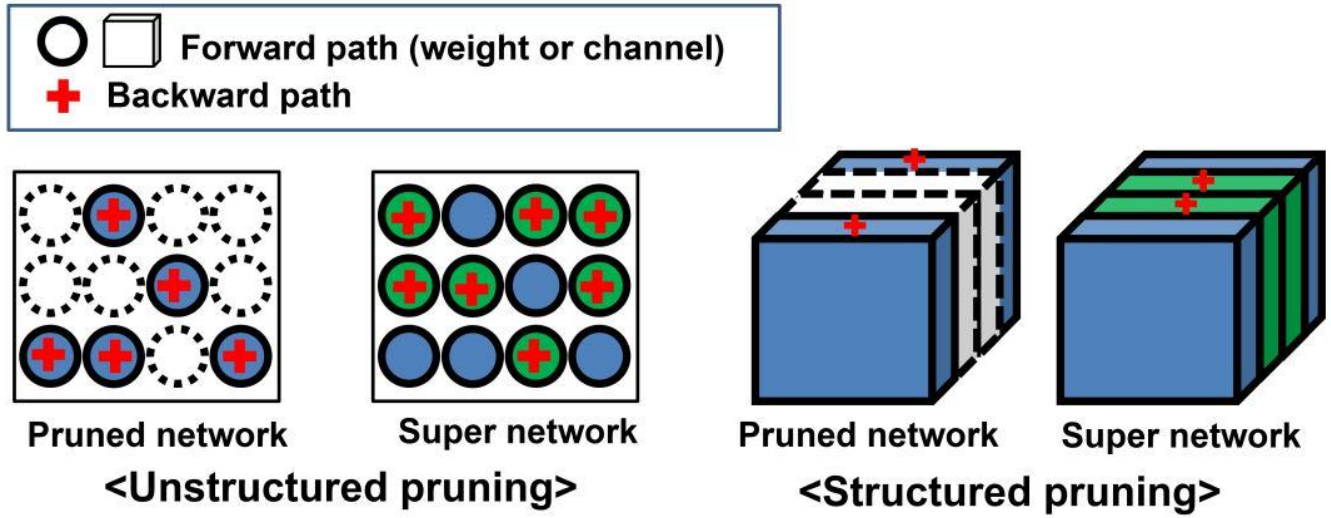
$$\mathbf{W} \leftarrow \mathbf{W} - \eta\{\mathbf{M} \odot \nabla_{\overline{\mathbf{W}}}\mathcal{L} + (1 - \mathbf{M}) \odot \nabla_{\mathbf{W}}\mathcal{L}\}$$

$$\nabla_{\overline{\mathbf{W}}}\mathcal{L} \triangleq \frac{\partial \mathcal{L} \circ \mathcal{F}^{\overline{\mathbf{W}}}(\mathbf{X})}{\partial \overline{\mathbf{W}}}, \quad \nabla_{\mathbf{W}}\mathcal{L} \triangleq \frac{\partial \mathcal{L} \circ \mathcal{F}^{\mathbf{W}}(\mathbf{X})}{\partial \mathbf{W}}$$



DCIL - Proposed method





DCIL – Experiments

Table 1: Top-1 test accuracy of various SOTA pruning methods on **CIFAR-10** for unstructured weight pruning. The numbers of SM, DSR and DPF methods are from Lin et al. (2020). The $\star$ means that the model does not converge. † indicates our numbers. DPF† is from using the official code and we report the last and best accuracy of both DPF† and our DCIL. We also report accuracy of the Dense† models which are not pruned at all. All experiments were conducted three times.

Model	Dense†	Methods							Target Pr. ratio
		SM (DZ, 2019)	DSR (MW, 2019)	DPF (Lin, 2020)	DPF† (Lin, 2020)		DCIL (Ours)		
					Last	Best	Last	Best	
ResNet-20	92.36 ± 0.10	89.76 ± 0.40	87.88 ± 0.04	90.88 ± 0.07	88.02 ± 0.12	90.87 ± 0.25	$\mathbf{91.61} \pm 0.20$	91.93 ± 0.11	90%
		83.03 ± 0.74	$\star$	88.01 ± 0.30	81.46 ± 2.26	87.84 ± 0.11	$\mathbf{90.54} \pm 0.19$	90.80 ± 0.23	95%
ResNet-32	93.22 ± 0.07	91.54 ± 0.18	91.41 ± 0.23	92.42 ± 0.18	91.14 ± 0.12	92.39 ± 0.09	$\mathbf{93.05} \pm 0.15$	93.23 ± 0.06	90%
		88.68 ± 0.22	84.12 ± 0.32	90.94 ± 0.35	86.52 ± 1.38	90.91 ± 0.45	$\mathbf{92.04} \pm 0.24$	92.27 ± 0.27	95%
ResNet-56	94.34 ± 0.19	92.73 ± 0.21	93.78 ± 0.20	93.95 ± 0.11	93.62 ± 0.14	93.97 ± 0.14	$\mathbf{94.16} \pm 0.08$	94.29 ± 0.05	90%
		90.96 ± 0.40	92.57 ± 0.09	92.74 ± 0.08	90.59 ± 0.38	92.81 ± 0.06	$\mathbf{93.75} \pm 0.14$	93.98 ± 0.12	95%
WideResNet-28-2	94.73 ± 0.03	93.41 ± 0.22	93.88 ± 0.08	94.36 ± 0.24	94.08 ± 0.34	94.32 ± 0.22	$\mathbf{94.69} \pm 0.13$	94.84 ± 0.28	90%
		92.24 ± 0.14	92.74 ± 0.17	93.62 ± 0.05	93.13 ± 0.24	93.61 ± 0.08	$\mathbf{94.01} \pm 0.21$	94.21 ± 0.04	95%
		85.36 ± 0.80	$\star$	88.92 ± 0.29	85.82 ± 0.45	88.77 ± 0.23	$\mathbf{91.19} \pm 0.16$	91.35 ± 0.24	99%

DCIL – Experiments

Table 3: Top-1 and Top-5 test accuracy of ResNet-18 and ResNet-50 on **ImageNet**. DPF† indicates our numbers using the official code. We report the best accuracy and the accuracy from the last epoch. Difference refers to difference between pruned and dense accuracy ('Last - Dense'). Best accuracy of Top-5 is selected based on Top-1 best epoch.

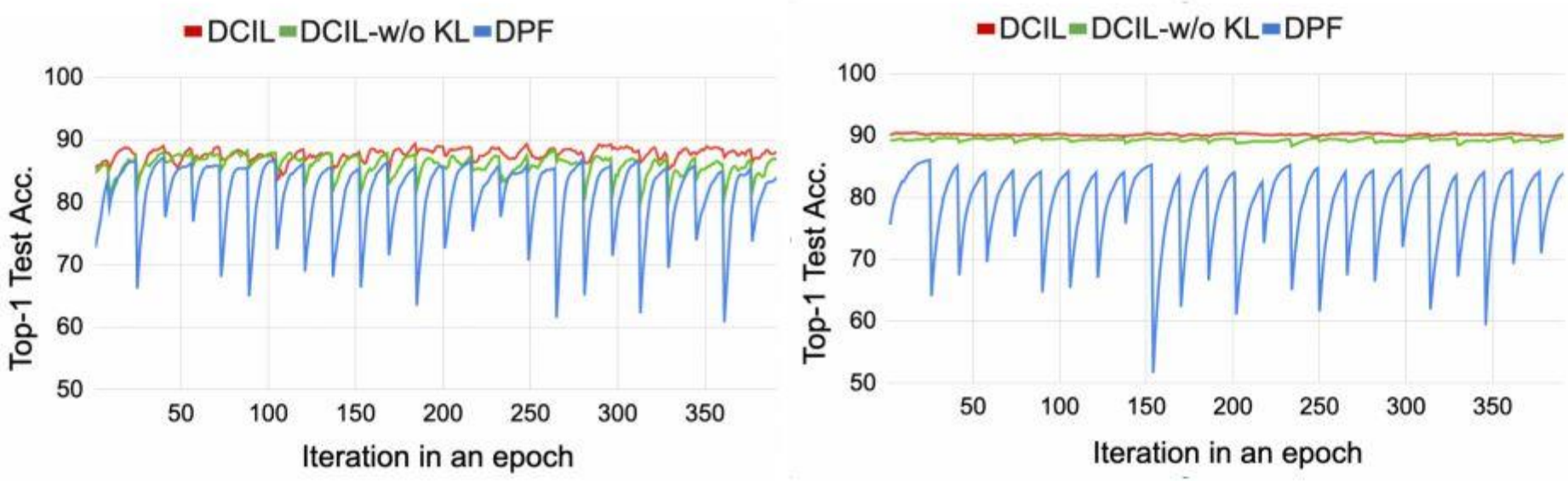
Model	Method	Top-1 accuracy				Top-5 accuracy				Target Pr. ratio
		Dense	Last	Best	Difference	Dense	Last	Best	Difference	
ResNet-18	DPF† (Lin, 2020)	70.50	69.08	69.19	-1.31	89.54	88.93	88.88	-0.66	80%
	DCIL (Ours)	70.50	**69.57**	69.62	-0.88	89.54	**89.26**	89.29	-0.25	80%
	DPF† (Lin, 2020)	70.50	67.37	67.66	-2.83	89.54	87.83	87.88	-1.66	90%
	DCIL (Ours)	70.50	**68.66**	68.66	-1.84	89.54	**88.57**	88.57	-0.97	90%
	DCIL w/o KL (Ours; $\lambda = 0$)	70.50	68.37	68.41	-2.09	89.54	88.32	88.26	-1.28	90%
ResNet-50	Incremental (ZG, 2017)	75.95	–	74.25	-1.70	92.91	–	91.84	-1.07	80%
	DSR (MW, 2019)	74.90	–	73.30	-1.60	92.40	–	92.40	0	80%
	SM (DZ, 2019)	75.95	–	74.59	-1.36	92.91	–	92.37	-0.54	80%
	DPF (Lin, 2020)	75.95	–	75.48	-0.47	92.91	–	92.59	-0.32	80%
	DPF† (Lin, 2020)	76.06	75.59	75.61	-0.47	92.85	92.66	92.71	-0.19	80%
	DCIL (Ours)	76.06	**76.16**	76.17	0.10	92.86	**92.94**	93.00	0.09	80%
	Incremental (ZG, 2017)	75.95	–	73.36	-2.59	92.91	–	91.27	-1.64	90%
	DSR (MW, 2019)	74.90	–	71.60	-3.30	92.40	–	90.50	-1.90	90%
	SM (DZ, 2019)	75.95	–	72.65	-3.30	92.91	–	91.26	-1.65	90%
	DPF (Lin, 2020)	75.95	–	74.55	-1.44	92.91	–	92.13	-0.78	90%
	DPF† (Lin, 2020)	76.06	74.39	74.48	-1.67	92.85	92.08	92.05	-0.77	90%
	DCIL (Ours)	76.06	**75.34**	75.40	-0.72	92.85	**92.58**	92.63	-0.27	90%
	DCIL w/o KL (Ours; $\lambda = 0$)	76.06	74.75	74.87	-1.31	92.85	92.31	92.24	-0.54	90%

DCIL – Experiments

Table 4: Top-1 test accuracy of our DCIL and other baseline methods on **CIFAR-10** for structured weight pruning. † indicates our numbers. We ran DPF† using the official code and official training schedule. Note that for a given pruning ratio DCIL and DPF prune filters across layers, meanwhile SFP prunes filters within the layer. Furthermore, we report the differences ('Pruned - Dense') in the Appendix. When comparing the differences, DCIL outperform with the higher gap in most cases. All reported numbers are averaged over three times.

Model	Dense†	SFP (H^+,2018)	DPF (Lin,2020)	DPF†		DCIL (Ours)		Target Pr. ratio
				Last	Best	Last	Best	
ResNet-32	93.22 ± 0.07	92.07 ± 0.22	92.18 ± 0.16	91.61 ± 0.15	91.81 ± 0.19	**93.02** ± 0.21	93.10 ± 0.21	30%
		91.14 ± 0.45	91.50 ± 0.21	90.01 ± 0.40	90.28 ± 0.40	**92.61** ± 0.07	92.76 ± 0.16	40%
WideResNet-28-2	94.73 ± 0.03	94.02 ± 0.24	94.52 ± 0.08	94.11 ± 0.17	94.30 ± 0.09	**94.55** ± 0.04	94.61 ± 0.08	40%
		$86.00 + 1.09$	$90.53 + 0.17$	$88.99 + 0.45$	$89.76 + 0.64$	**92.10** $+ 0.11$	$92.32 + 0.15$	80%
WideResNet-28-4	95.50 ± 0.07	95.15 ± 0.11	95.50 ± 0.05	95.36 ± 0.16	95.52 ± 0.14	95.46 ± 0.06	95.59 ± 0.06	40%
		91.88 ± 0.59	93.79 ± 0.09	93.41 ± 0.07	93.51 ± 0.11	**94.18** ± 0.04	94.38 ± 0.08	80%
WideResNet-28-8	95.93 ± 0.07	95.62 ± 0.04	96.06 ± 0.12	95.86 ± 0.17	96.03 ± 0.18	95.73 ± 0.07	95.91 ± 0.10	40%
		94.22 ± 0.21	95.15 ± 0.03	94.97 ± 0.20	95.16 ± 0.12	**95.43** ± 0.09	95.49 ± 0.05	80%

DCIL – Experiments



(a) 120th epoch (b) 200th epoch

Figure 5: Top-1 test accuracy of every iteration in an epoch. To show the effectiveness of changes in active weights caused by changes in pruning masks, we evaluated ResNet-20 pruned with 95% sparisty using Red: DCIL, Green: DCIL without KL loss, and Blue: DPF on CIFAR10 every iteration at (a) 120th epoch and (b) 200th epoch. The accuracy of all methods shows a repetitive pattern with a pruning frequency (**F**) of 16.

DCIL – Conclusion

- To make diverse sparse patterns, reviving inactive weights with coarse gradients using STE has been used
- This causes instability during training and performance degradation due to the gradient approximation
- we propose a novel Dynamic Collective Intelligence Learning (DCIL) which finds a sparse model by training inactive weights with refined gradients rather than using approximated coarse gradients